

Resolviendo EMT

CyPLP 2023

EMT-3-A

Verdadero o Falso

Marque con una cruz las respuestas falsas.

- ☐ En C, si asignamos a una variable puntero, la dirección de otra variable, nunca estamos seguros que la variable puntero nunca contendrá referencias sueltas.
- ☒ El componente principal del paradigma funcional es la transparencia referencial
- ☒ La forma de contener una excepción es una de las preguntas que debemos respondernos cuando evaluamos este concepto (excepciones) en un lenguaje de programación.

Mi resolución esta aca:

Primer respuesta: Verdadera.

Segunda respuesta: Falsa (para mi, lo principal del paradigma funcional es que esta basado en funciones matematicas)

Tercer respuesta: Verdadera.

Pregunta

```
1  #include <stdio.h>
2  int main()
3  {
4  {
5  static int i;
6  int res=0;
7  bool modificado=false;
8  leer1A10(i);///lee del teclado el valor numerico de i del 1 al 10
9  switch(i){
10 case 1: res=1;modificado=true;break;
11 case 2: if(modificado==false)res=2;modificado=true;break;
12 case 3: if(modificado==false)res=3;modificado=true;break;
13 case 4: if(modificado==false)res=4;modificado=true;break;
14 case 5: if(modificado==false)res=5;modificado=true;break;
15 }
16
17 return 0;
18 }
```

Lea el siguiente fragmento de código en C y determine qué situaciones pueden presentarse en el uso de la sentencia de selección. Indique qué particularidades presenta esta sentencia en este caso en particular.

Respuesta *ver 00*

En este código se asume que se lee un número del 1 al 10. Si el número está entre 1 y 5 inclusive, se entra por alguno de los `case` y se modifican las variables `res` y `modificado`. Además, el uso de `break` evita que se ejecuten los `case` siguientes, lo cual es correcto.

Pero, si se lee un número entre 6 y 10, no hay ningún `case` que lo contemple, por lo que el código no realiza ninguna acción, lo cual puede ser una falla lógica (no sabemos el

proposito de este codigo porque no lo desarrollamos nosotros). Ademas existe el riesgo de ingresar un carácter no numérico.

Para contemplar todos los posibles casos no manejados explícitamente, lo ideal es agregar una cláusula default al switch, de forma que se asegure una acción definida para cualquier valor no considerado.

```
1 <?php
2 try{
3     $a=0;
4     $b=4;
5     $c=$b/$d;
6 }
7 catch(DivisionByZeroError $e){
8     print_r("dividió por 0");
9 }
10 finally{
11     print_r("SE PRODUJO UN ERROR EN LA DIVISIÓN");
12 }
13
14 ?>
```

y name

Pregunta

Sea el siguiente código escrito en PHP indique de qué forma evitaría un error en el sistema si se pudieran manejar cualquiera de las excepciones que se que no se contemplan en este código. Justifique indicando cual es el resultado de la ejecución.

Respuesta

la forma en la
que se evitaria

En este codigo solamente se estarían manejando las excepciones por division por cero, las cuales serían manejadas por la clausula catch(DivisionByZeroError \$e). Pero en el caso de que ocurra otra excepcion, no estaria contemplada en este codigo, por lo que la solucion para manejar cualquier otro tipo de excepcion es agregar una clausula nueva catch (excepcion \$ex) y manejarla como quisieramos.

Con respecto al resultado de la ejecucion teniendo en cuenta el codigo original:

Se inicializan las variables a=0 y b=4, se intenta dividir, si d=0 lanzaria la excepcion DivisionByZeroError, la cual seria manejada por que tenemos la clausula catch, pero si d tuviera otro valor, por ejemplo un valor no numerico, se lanzaria una excepcion que no estaria contemplada en el codigo original y rompería el programa, mientras que si agregamos la clausula catch generica, estaríamos manejando ese tipo de excepciones.

Pregunta

<pre>fact2 :: Integer -> Integer fact2 0 = 1 fact2 n = n * fact2(n-1)</pre> funcional	<pre>function fact(var n: integer): int; begin if (n = 0) then fact := 1 else fact := n * fact(n - 1); end;</pre>
---	---

Sean los siguientes, códigos escritos en distintos paradigmas. indique a qué paradigma pertenece cada uno y qué característica de seguridad representa uno respecto del otro. Justifique la respuesta.

El primer codigo esta escrito en paradigma funcional, el cual esta basado en funciones matematicas puras.

Características de seguridad:

- Las variables son inmutables, lo que significa que una vez asignado un valor no puede ser modificado, esto reduce el riesgo de efectos secundarios

- La transparencia referencial es una característica clave, una función siempre devuelve el mismo resultado para los mismos argumentos.

El segundo código está escrito en paradigma imperativo, donde el enfoque principal es indicar una secuencia de instrucciones que modifican el estado del programa

Características de seguridad:

- Las variables son mutables y su valor puede cambiar a lo largo del tiempo, lo cual puede generar errores si no se usan correctamente.
- La mutabilidad y el acceso a variables globales pueden hacer que el sistema sea más propenso a errores.